Gazebo and PX4 for the Atlas 10 - EDF airframe

From nothing to a hover in SITL and HITL, with the tiltlab stack

tiltlab project notes

2026-09-11

This guide assumes you have never used Gazebo or PX4. It is written so that, with this document and an internet connection, you can rebuild and operate everything the repository does without further help. Where an official page explains something better than a rewrite would, the guide sends you there and tells you what to take from it. Where this project learned something the official pages do not say, the guide says it in full.

Contents

1 The map: what talks to what

Everything here is three programs and one cable.

1. **A physics simulator** (Gazebo) integrates rigid-body motion of the aircraft, spins simulated fans that produce thrust, and produces fake sensor readings.
2. **A flight controller** (PX4) reads sensors, estimates attitude and position, runs the controllers and the *control allocator* that turns "roll torque, pitch torque, yaw torque, thrust" into ten fan commands, and sends those commands back to the simulator.
3. **tiltlab** (this repository) describes the aircraft (a *scenario*: fan positions, fan thrust axes, mass, inertia, foil deflections), computes whether it can hover and how much control authority it has, and *exports* the simulator model and the PX4 configuration so that the two agree exactly.
4. **The cable** is the difference between SITL and HITL:
   - **SITL** (software in the loop): PX4 runs as a program on the same PC as Gazebo. No hardware. Fast to iterate.
   - **HITL** (hardware in the loop): PX4 runs on the real Pixhawk, connected by USB. Gazebo replaces the aircraft and its sensors. The controller code, its timing and its parameters are exactly what will fly.

```
scenario JSON -> tiltlab -> exported harness -> WSL launcher -> Gazebo <-> PX4 (PC or
    Pixhawk)
```

Why this matters here: Every failure in this project was a disagreement between two of those boxes: the model and the allocator disagreed about geometry, the simulator and the board disagreed about attitude, or the parameter file and the board disagreed about integers. Keep asking "which two boxes disagree?"

2 Gazebo, the minimum you need

2.1 Two Gazebos

There are two unrelated programs called Gazebo:

- **Gazebo Classic** (versions 9, 10, 11; command `gazebo`). Old, stable, and the only one PX4 v1.17 speaks HITL through. We run it on Ubuntu 22.04.
- **gz sim** (formerly Ignition; versions Fortress, Garden, Harmonic; command `gz sim`). The current one. PX4's SITL bridge targets it. We run Harmonic on Ubuntu 24.04.

They have different file layouts, plugin systems and command-line tools. Do not mix their documentation. Official entry points:

- gz sim: https://gazebosim.org/docs/harmonic/getstarted/ (read "Getting started" and "Understanding the GUI"; skip ROS integration).
- Gazebo Classic: https://classic.gazebosim.org/tutorials (read "Beginner: Overview" and "Build a robot: Model structure and requirements").

2.2 SDF: the file that describes everything

Both Gazebos read **SDF** (Simulation Description Format), an XML dialect. Reference: http://sdformat.org/spec. You need four concepts:

world
: The scene: gravity, ground plane, light, which models to spawn and where. One file, `*.world` (Classic) or `*.sdf` (gz sim). Our exporter writes one per scenario.

model
: The aircraft. A directory with `model.config` (name, author) and `model.sdf` (the content).

Gazebo finds models through the `GAZEBO_MODEL_PATH` (Classic) or `GZ_SIM_RESOURCE_PATH` (gz sim) environment variable; the launcher sets these.

link A rigid body inside a model: mass, inertia, collision shape, visual shape, pose. Our model has `base_link` (the airframe, mass and inertia from the scenario), ten `rotor_N` links (one per fan) and an IMU link.

joint
Connects two links. Each rotor joint is a *revolute* joint whose axis is the fan's spin axis.

Poses are "x y z roll pitch yaw" in metres and radians, in the *parent's* frame. A rotor link's pose therefore places the fan on the airframe and points it.

2.3 Frames: the single most important paragraph

Gazebo's body frame is **FLU**: x forward, y left, z up. PX4's is **FRD**: x forward, y right, z down. Converting is a 180° rotation about x: $(x, y, z)_{\text{FRD}} \rightarrow (x, -y, -z)_{\text{FLU}}$. Gazebo's world frame is **ENU** (x east, y north, z up); PX4's is **NED**. A vehicle facing Gazebo's $+x$ (east) has PX4 heading 090°. That is not a bug; you will see "yaw 90" on a level, still vehicle and it is correct.

tiltlab works in FRD internally and applies the FLU conversion once, when writing the model. If you ever hand-edit a pose, do it in FLU.

Why this matters here: The airframe hovers at 25° nose-up. tiltlab's scenario field `frame.hover_pitch_deg` rotates the airframe geometry into the flight-controller frame, so the exported model's `base_link` is level when the real airframe is at 25°. In Gazebo you therefore *see* the airframe mesh tilted 25° while PX4 reads "level". That is the intended design, in both simulators.

2.4 Plugins

A plugin is a shared library Gazebo loads for a model. Ours (Classic, from PX4's `sitl_gazebo-classic`):

- `libgazebo_motor_model.so`, one per rotor. Thrust $= k_T\,\omega^2$ along the rotor link's $+z$, torque $= k_M \cdot$ thrust, with a spin-up time constant. The exporter sets k_T from the fan curve and the time constant from the fan lag.
- `libgazebo_imu_plugin.so`, `_magnetometer_`, `_barometer_`, `_gps_`: simulated sensors.
- `libgazebo_mavlink_interface.so`: the bridge. Reads sensors, sends them to PX4 as MAVLink; receives PX4's motor commands and publishes them to the motor plugins. Its parameters that matter: `serialEnabled`, `serialDevice` (HITL), `hil_mode`, `hil_state_level`, `input_scaling` per motor channel.

gz sim's equivalents live in PX4's `Tools/simulation/gz` and the `gz_bridge` module inside PX4 itself; PX4 subscribes to gz topics directly, no MAVLink in between.

2.5 Topics and tools

Gazebo publishes on internal topics. Useful commands (Classic):

```
gz topic -l # list topics (note the world name in each path)
gz topic -e <topic> # echo (fails silently on PX4's custom message types)
gz stats # real-time factor: keep it at 1.00 for HITL
gz world -w <world> --reset-all # put every model back where it spawned
gz model -w <world> -m <name> -p # pose; -w is required unless the world is called "
    default"
```

gz sim: `gz topic -l`, `gz topic -e -t <topic>`, `gz model -list`.

3 PX4, the minimum you need

Official documentation: https://docs.px4.io/v1.17/en/ (we pin v1.17.0). Read, in this order: "Basic Concepts", "Flight Modes (Multicopter)", "Parameters and Configurations", "Control

Allocation", "Simulation" overview, then "Hardware in the Loop".

3.1 Parameters

PX4 is configured by named parameters, e.g. `MC_ROLLRATE_P`. Each has a type: INT32 or FLOAT. They live on the board (or in `parameters.bson` for SITL) and survive reboots once saved. Three things this project learned the hard way:

1. Over MAVLink, **INT32 parameters travel as raw bits in a float field**. Writing them as ordinary floats corrupts every integer parameter (`HIL_ACT_FUNC1` became 1120534528, which is the bit pattern of $101.0f$), and reading them back the same wrong way agrees with itself, so nothing looks wrong. Write integers through the board's shell (`param set NAME VALUE`) or use `scripts/px4_board.py`, which does that and decodes reads correctly.
2. Some parameters are read **only at boot**: `SYS_HITL`, `SYS_AUTOSTART`, `EKF2_EN`, `SDLOG_*`. Change, save, reboot, then check.
3. An airframe file's `param set-default` lines never override a value the user saved; `param set` lines do. `1001_rc_quad_x.hil` uses `set-default` for its 4-rotor geometry, which is why our saved 10-rotor set survives it.

3.2 The control allocator

PX4 does not know about "quad" or "hexa"; it knows a matrix. For each rotor i you give a position `CA_ROTORi_PX/PY/PZ` (m, FRD, from the centre of gravity), a thrust axis `CA_ROTORi_AX/AY/AZ` (unit vector, FRD), a thrust coefficient `CA_ROTORi_CT` and a torque coefficient `CA_ROTORi_KM`. PX4 builds the effectiveness matrix (source: `ActuatorEffectivenessRotors.cpp`), pseudo-inverts it and desaturates (`ControlAllocationSequentialDesaturation.cpp`). tiltlab replicates that code line for line so its predictions match what PX4 will do; the exported parameters are the same numbers the replica used. Read "Control Allocation" in the docs once; then trust the exporter.

3.3 Gains that matter for this airframe

The rate loop (`MC_ROLLRATE_P/I/D` etc.), the attitude loop (`MC_ROLL_P`...), the position loops (`MPC_XY_*`, `MPC_Z_*`), the hover throttle `MPC_THR_HOVER`, the tilt limit `MPC_TILTMAX_AIR`, and `MC_AT_EN` (autotune; **keep it 0**, autotune produced gains three to four times too high and every axis rang). tiltlab sizes the first three from authority, inertia and fan lag; the rest were tuned in SITL and saved in the scenario under `control.px4_params_override`, from where both exports pick them up.

4 Windows, WSL2 and USB

Both simulators run in WSL2 (Windows Subsystem for Linux). Two distributions, because gz sim Harmonic wants Ubuntu 24.04 and Gazebo Classic 11 exists only up to 22.04:

```
wsl --list --verbose # Ubuntu-24.04 (SITL), Ubuntu-22.04 (HITL)
wsl -d Ubuntu-22.04 -- bash -lc "<command>"
```

The repository is reachable inside both as `~/utopia/vibe-coded` (a symlink; the space in "Utopia Labs" breaks quoting through `wsl.exe`, so never pass the `/mnt/c/...` spelling on a command line).

GUI: WSLg shows Linux windows on the Windows desktop. If every window is grey and titled `[WARN:COPY MODE]`, WSLg lost its shared-memory channel at start-up; only `wsl -shutdown` from PowerShell fixes it (it closes every distribution).

USB into WSL uses usbipd-win (https://github.com/dorssel/usbipd-win):

```
usbipd list # find the Pixhawk's BUSID, e.g. 2-4
usbipd bind --busid 2-4 # once, administrator PowerShell
usbipd attach --wsl Ubuntu-22.04 --busid 2-4 # each session (--auto-attach to survive
    reboots)
usbipd detach --busid 2-4 # give it back to Windows (QGroundControl)
```

Inside WSL the board is `/dev/ttyACM0`, owned by `root:dialout`; your user must be in `dialout`. A board reboot re-enumerates and often comes back as `ttyACM1`; the launcher handles either. The distribution must be running for `attach` to work (keep a shell open).

5 Installing the stack from scratch

Do these once. Each step's check is written next to it.

1. **Windows tools**: `uv` (Python), Node, `make`, MiKTeX (for this guide), usbipd-win. In the repo: `source scripts/env.sh` (bash) or `. .\scripts\env.ps1` puts them on `PATH`. Check: `make test` is green.
2. **WSL distributions**: `wsl -install -d Ubuntu-24.04` and `wsl -install -d Ubuntu-22.04`. Create the symlink in each: `mkdir -p ~/utopia && ln -s "/mnt/c/Users/<you>/Documents/Utopia Labs/vibe-coded" ~/utopia/vibe-coded`.
3. **PX4 source** in each distribution, pinned:

   ```
   git clone --recursive -b v1.17.0 https://github.com/PX4/PX4-Autopilot.git ~/PX4-
       Autopilot
   cd ~/PX4-Autopilot && bash ./Tools/setup/ubuntu.sh # toolchains; do NOT pass --no-
       nuttx in 22.04
   ```

   Check (22.04): `arm-none-eabi-gcc -version`. Check (24.04): `gz sim -version` prints Harmonic 8.x. In 22.04 also `sudo apt install gazebo libgazebo-dev`.
4. **Let the launcher finish the rest**. Run `bash ~/utopia/vibe-coded/scripts/wsl/tiltlab_gazebo.sh -check` in each distribution. It lists every tool with its version, builds what is missing when asked, and applies the PX4 patches a 10-motor vehicle needs (PX4's gz bridge assumes at most 8 ESCs). Repeat until every line says `[ok]`.
5. **HITL firmware** (22.04, once). Stock `px4_fmu-v6x_default` has no `pwm_out_sim` and adding it overflows flash by 22 884 bytes, so the exporter ships a board label that also drops the fixed-wing and VTOL modules:

   ```
   bash ~/utopia/vibe-coded/scripts/wsl/tiltlab_gazebo.sh --build-firmware --harness <
       hitl harness dir>
   ```

   It copies `fmu-v6x_hitl.px4board`, runs `make px4_fmu-v6x_hitl` (94.6 % of flash) and offers the upload. Check on the board: `pwm_out_sim status` lists channels 0..9 with functions 101..110 after the parameters are loaded.

6 Flying SITL

1. In the app (`make dev`, http://127.0.0.1:8000) load or build a scenario, check *hover: exact* and the authority numbers, Save.
2. Metrics panel, *Launch Gazebo*, **SITL**. The app exports `exports/gazebo/<name>/` (model, world, airframe `NNNN_gz_<name>`, README) and starts the launcher in Ubuntu-24.04. Its console tail appears under the buttons; the full console is `exports/logs/gazebo_sitl.log`. Without the browser: `make menu`, option 2.
3. Wait for `Ready for takeoff!` plus about ten seconds (height estimate), then

   ```
   wsl -d Ubuntu-24.04 -- bash ~/utopia/vibe-coded/scripts/wsl/px4ctl.sh takeoff
   ```

```
wsl -d Ubuntu-24.04 -- bash ~/utopia/vibe-coded/scripts/wsl/px4ctl.sh land
wsl -d Ubuntu-24.04 -- bash ~/utopia/vibe-coded/scripts/wsl/px4ctl.sh log # copies
    the newest ulog
uv run --project backend python scripts/hover_report.py exports/logs/<file>.ulg
```

4. Good hover on this airframe: attitude under 1° peak to peak, position under 0.3 m, torque demand well below 1. Tune live with `px4ctl.sh param NAME VALUE ...`, land and take off again so integrators restart, and save winners in the scenario's `control.px4_params_override`.

What SITL cannot tell you: PX4-SITL's clock *is* Gazebo's clock (lockstep), so the flight stack never sees late sensor data. The real board runs on its own clock. If a geometry only just hovers in SITL, expect HITL to be worse.

7 Flying HITL

7.1 How HITL differs, in this project's configuration

The board is the flight controller; Gazebo Classic is the aircraft. Two designs exist for how the board learns its attitude:

- `hil_state_level 0`: Gazebo sends simulated IMU, magnetometer, barometer and GPS (`HIL_SENSOR`, `HIL_GPS`); the board's own EKF estimates attitude. The official default.
- `hil_state_level 1`: Gazebo sends its ground-truth attitude, position and velocity (`HIL_STATE_QUATERNI`
  the board uses that directly.

This project uses level 1, and the exported parameter set carries what that needs: `EKF2_EN 0` (otherwise the EKF and the HIL handler both publish attitude and the values flip between them), `SYS_HAS_MAG 0`, `SYS_HAS_BARO 0` (no such sensors are sent at level 1), and calibration slot 0 pointed at the simulated IMU (device id 1310988) so the arming checks stop looking for the real ICM. The reason for leaving level 0: the Classic IMU plugin handed this board an accelerometer tilted about 25° while the model was level, also with PX4's own `iris_hitl` reference model, and the EKF believed it. That was never resolved at the source; level 1 sidesteps it, and what HITL is for (the controller and allocator running on real hardware against the real geometry) is fully exercised.

7.2 Session checklist

1. Fans and ESCs unpowered. Board on USB, attached to WSL (Section 4). QGroundControl closed (one program owns the serial port).
2. Parameters onto the board, once per scenario (no sim running):

```
python3 scripts/px4_board.py --dev /dev/ttyACM0 push exports/gazebo_hitl/<name>
    _hitl/px4/<name>_hitl.params
python3 scripts/px4_board.py --dev /dev/ttyACM0 shell reboot
python3 scripts/px4_board.py --dev /dev/ttyACM0 verify exports/gazebo_hitl/<name>
    _hitl/px4/<name>_hitl.params
```

   `verify` must print `N of N match`.
3. App, *Launch Gazebo*, **HITL** (or `make menu`, option 3). The launcher installs the model into the PX4 Classic tree, finds the serial device, starts the QGC relay (WSL2 drops UDP broadcast, so QGroundControl on Windows would otherwise never see the vehicle) and opens Gazebo.
4. Before arming, confirm the board agrees with the sim:

```
python3 scripts/px4_board.py status # HIL flag True, roll/pitch near 0, velocities
    0, Preflight check: OK
```

   (default link is the sim's SDK port 14540, so this works while Gazebo owns the serial.)

5. Arm and fly from the transmitter or QGroundControl. Afterwards `python3 scripts/px4_board.py pull-log exports/logs` then `hover_report.py`. Logs land in `/fs/microsd/log/sessNNN/` because the board has no time source at level 1.

7.3 Returning the board to flight

Reflash `px4_fmu-v6x_default`, load your flight parameter file, confirm `SYS_HITL` 0, `EKF2_EN` 1, `SYS_HAS_MAG` 1, `SYS_HAS_BARO` 1, re-select your airframe and recalibrate the IMU (the HITL set points calibration slot 0 at the simulated device). On the real aircraft the IMU is bolted to the airframe and reads the 25° hover pitch; `SENS_BOARD_Y_OFF` (degrees) makes PX4's body frame the hover frame. It acts on real sensors only, so it plays no part in HITL.

8 When it goes wrong

You see	It means	Do
Motors stay at zero after takeoff; allocator reports thrust unallocated	the geometry has no hover trim (tiltlab shows *Fz unattainable*)	pick a feasible sweep row or change hover pitch
Lifts, then flips within two seconds	PX4 default gains on a 12 kg, 150 ms-fan airframe	gains from the exporter (`px4_tuning`); never Autotune
Slow 0.25 Hz roll/pitch swing to the tilt limit, metres of wander	position loop faster than attitude loop	the exporter's `MPC_*` scaling; lower `MPC_TILTMAX_AIR` on thin margin
Yaw torque demand above 1	yaw axis has almost no authority; gains sized too stiff	soften `MC_YAWRATE_P/I`, `MC_YAW_P`
Translates away at speed, altitude fine	a standing trim moment the integrator cannot hold	raise `MC_PR_INT_LIM` (pitch) with a larger `MC_PITCHRATE_I`
Second Gazebo launch shows a grey or duplicate window	a `gzserver` from the last run holds port 11345	`tiltlab_gazebo.sh -stop` (the launcher also asks)
`[WARN:COPY MODE]` window	WSLg lost shared memory	`wsl -shutdown`, relaunch
`Error opening serial device: set_option: Input/output error`, `Tx queue overflow`	the board re-enumerated as another `ttyACM`	relaunch; the launcher picks the one present
Board attitude far from the sim's, parked; arms into a lunge	sim and board disagree about attitude	`hil_state_level` 1 + `EKF2_EN` 0 (in the export); compare with `px4_board.py status` before arming
`Preflight Fail: Accel Sensor 0 missing`, `barometer 0 missing`, `Found 0 compass`	sensor-presence checks at state level 1	`SYS_HAS_MAG/BARO` 0, calibration slot 0 = 1310988 (in the export)
`Flight termination active`	it flipped past 60°; latched	reboot the board (`px4_board.py shell reboot`); `CBRK_FLIGHTTERM` 121212 disables it for bench work

You see	It means	Do
`logger: not running`, no log of the flight	`SDLOG_BACKEND 0` on the board	`SDLOG_MODE 2`, `SDLOG_BACKEND 1` (in the export), reboot
Integer parameters read as huge numbers or 0 after a MAVLink load	float write of INT32	`px4_board.py push` (shell) and `verify`
QGroundControl "Disconnected" while Gazebo runs	WSL2 dropped the broadcast	the relay must be running (launcher output); manual UDP link: listen 14551, server `<WSL ip>:18570`

9 Parameters used in this project

Every parameter touched during the simulation and tuning phase, with what it does (from the pinned PX4 v1.17.0 source: `PARAM_DEFINE` doc blocks and `module.yaml`), the range or enumeration PX4 accepts, and the value flown on the 50/55/60/65 geometry at 25° hover pitch. Indexed families (`CA_ROTORn_*`, `HIL_ACT_FUNCn`, `SIM_GZ_EC_*n`) are shown once with rotor 0's value. Values marked "hand tuned" live in the scenario's `control.px4_params_override`; the rest are computed or fixed by the exporter.

Parameter	What it does	Allowed	Used
Geometry and allocation (from the scenario; identical in SITL and HITL)			
`CA_AIRFRAME`	Airframe selection	0 = Multirotor; 1 = Fixed-wing; 2 = Standard VTOL; 3 = Tiltrotor VTOL; 4 = Tailsitter VTOL; 5 = Rover (Ackermann); 6 = Rover (Differential); 7 = Motors (6DOF); 8 = Multirotor with Tilt; 9 = Custom; 10 = Helicopter (tail ESC); 11 = Helicopter (tail Servo); 12 = Helicopter (Coaxial); 13 = Rover (Mecanum); 14 = Spacecraft 2D; 15 = Spacecraft 3D	0
`CA_METHOD`	Control allocation method	0 = Pseudo-inverse with output clipping; 1 = Pseudo-inverse with sequential desaturation technique; 2 = Automatic	2
`CA_ROTOR_COUNT`	Total number of rotors	0 = 0; 1 = 1; 2 = 2; 3 = 3; 4 = 4; 5 = 5; 6 = 6; 7 = 7; 8 = 8; 9 = 9; 10 = 10; 11 = 11; 12 = 12	10
`CA_ROTORn_PX`	Position of rotor n along X body axis relative to center of gravity	-100 to 100 m	-0.0996811

Parameter	What it does	Allowed	Used
CA_ROTORn_AX	Axis of rotor n thrust vector, X body axis component	-100 to 100	0.258819
CA_ROTORn_CT	Thrust coefficient of rotor n	0 to 100	31.45
CA_ROTORn_KM	Moment coefficient of rotor n	-1 to 1	0
THR_MDL_FAC	Thrust to motor control signal model parameter	0 to 1	1
Rate loop (sized by px4_tuning; yaw and integrator limits hand tuned)			
MC_ROLLRATE_P	Roll rate P gain	0.01 to 0.5	0.3898
MC_ROLLRATE_I	Roll rate I gain	0 to ...	0.2339
MC_ROLLRATE_D	Roll rate D gain	0 to 0.01	0.01949
MC_ROLLRATE_K	Roll rate controller gain	0.01 to 5	1
MC_RR_INT_LIM	Roll rate integrator limit	0 to ...	0.3
MC_PITCHRATE_P	Pitch rate P gain	0.01 to 0.6	0.6
MC_PITCHRATE_I	Pitch rate I gain	0 to ...	0.5
MC_PITCHRATE_D	Pitch rate D gain	0 to ...	0.03
MC_PR_INT_LIM	Pitch rate integrator limit	0 to ...	0.6
MC_YAWRATE_P	Yaw rate P gain	0 to 0.6	0.08
MC_YAWRATE_I	Yaw rate I gain	0 to ...	0.02
MC_YAWRATE_D	Yaw rate D gain	0 to ...	0
MC_YR_INT_LIM	Yaw rate integrator limit	0 to ...	0.15
Attitude loop and autotune			
MC_ROLL_P	Roll P gain	0 to 12	0.762
MC_PITCH_P	Pitch P gain	0 to 12	0.762
MC_YAW_P	Yaw P gain	0 to 5	0.25
MC_AT_EN	Multicopter autotune module enable	0 off, 1 on	0
MC_AT_APPLY	Controls when to apply the new gains	0 = Do not apply the new gains (logging only); 1 = Apply the new gains after disarm; 2 = WARNING Apply the new gains in air	0
Position and velocity loops, hover throttle, limits (hand tuned in SITL)			
MPC_THR_HOVER	Vertical thrust required to hover	0.1 to 0.8 norm	0.384
MPC_XY_P	Proportional gain for horizontal position error	0 to 2	0.25
MPC_XY_VEL_P_ACC	Proportional gain for horizontal velocity error	1.2 to 5	0.4
MPC_XY_VEL_I_ACC	Integral gain for horizontal velocity error	0 to 60	0.06
MPC_XY_VEL_D_ACC	Differential gain for horizontal velocity error	0.1 to 2	0.15
MPC_XY_VEL_MAX	Maximum horizontal velocity	0 to 20 m/s	1
MPC_Z_P	Proportional gain for vertical position error	0.1 to 1.5	0.4
MPC_Z_VEL_P_ACC	Proportional gain for vertical velocity error	2 to 15	1.2
MPC_Z_VEL_I_ACC	Integral gain for vertical velocity error	0.2 to 3	0.6
MPC_Z_VEL_MAX_UP	Maximum ascent velocity	0.5 to 8 m/s	1
MPC_Z_VEL_MAX_DN	Maximum descent velocity	0.5 to 4 m/s	0.7
MPC_TILTMAX_AIR	Maximum tilt angle in air	20 to 89 deg	8
MPC_TKO_SPEED	Takeoff climb rate	1 to 5 m/s	1
MPC_ACC_HOR	Acceleration for autonomous and for manual modes	2 to 15 m/sˆ2	1

Parameter	What it does	Allowed	Used
MPC_ACC_UP_MAX	Maximum upwards acceleration in climb rate controlled modes	2 to 15 m/s^2	2
MPC_ACC_DOWN_MAX	Maximum downwards acceleration in climb rate controlled modes	2 to 15 m/s^2	2
MPC_JERK_AUTO	Jerk limit in autonomous modes	1 to 80 m/s^3	1
MIS_TAKEOFF_ALT	Default take-off altitude	0 to ... m	2.5 (default)
SITL only (gz sim bridge; written to the airframe file)			
SIM_GZ_EC_FUNCn	Output function driven by gz ESC channel n (mixer_module output functions)	0 disabled; 101..112 Motor1..Motor12	101..110
SIM_GZ_EC_MINn	Command sent to gz at zero thrust (rotor rad/s)	0 to 1000	0
SIM_GZ_EC_MAXn	Command sent to gz at full thrust (rotor rad/s)	0 to 1000	1000
SIM_GZ_EN	Simulator Gazebo bridge enable	0 off, 1 on	1
MAV_0_BROADCAST	Broadcast heartbeats on local network for MAVLink instance n	0 = Never broadcast; 1 = Always broadcast; 2 = Only multicast	1
HITL only (written to the .params file loaded on the board)			
SYS_HITL	Enable HITL/SIH mode on next boot	-1 = external HITL; 0 = HITL and SIH disabled; 1 = HITL enabled; 2 = SIH enabled	1
SYS_AUTOSTART	Auto-start script index	airframe id; 1001 = HIL Quadcopter X	1001
HIL_ACT_FUNCn	Output function on pwm_out_sim channel n, sent in HIL_ACTUATOR_CONTROLS	0 disabled; 101..112 Motor1..Motor12	101..110
CBRK_SUPPLY_CHK	Circuit breaker for power supply check	0 to 894281	894281
UAVCAN_ENABLE	UAVCAN mode	0 = Disabled; 1 = Sensors Manual Config; 2 = Sensors Automatic Config; 3 = Sensors and Actuators (ESCs) Automatic Config	0
EKF2_EN	EKF2 enable	0 off, 1 on	0
SYS_HAS_MAG	Control if and how many magnetometers are expected	0 off, 1 on	0
SYS_HAS_BARO	Control if the vehicle has a barometer	0 off, 1 on	0
CAL_ACC0_ID	Accelerometer n calibration device ID	device id; 1310988 = the HIL receiver's SIM IMU	1310988
CAL_GYRO0_ID	Gyroscope n calibration device ID	device id; 1310988 = the HIL receiver's SIM IMU	1310988
CAL_ACC0_*OFF	Accelerometer offsets of slot 0 (X, Y, Z)	any (m/s^2)	0
CAL_GYRO0_*OFF	Gyroscope offsets of slot 0 (X, Y, Z)	any (rad/s)	0
CAL_ACC0_*SCALE	Accelerometer scale of slot 0 (X, Y, Z)	0.1 to 3	1

Parameter	What it does	Allowed	Used
SDLOG_MODE	Logging Mode	0 = when armed until disarm (default); 1 = from boot until disarm; 2 = from boot until shutdown; 3 = while manual input AUX1 >30%; 4 = from 1st armed until shutdown	2
SDLOG_BACKEND	Logging Backend (integer bitmask)	0 to 3	1
GPS_1_CONFIG	Serial port for GPS 1; rcS sets 0 in HITL (no GPS)	0 disabled; a serial port id	0 (set by rcS)
Bench and real-aircraft settings mentioned in the text			
CBRK_FLIGHTTERM	Circuit breaker for flight termination	0 to 121212	0 (kept)
SENS_BOARD_Y_OFF	Board rotation Y (pitch) offset	-45 to 45 deg	0 in HITL; 25 on the real aircraft (sign to verify)
SENS_BOARD_ROT	Coarse board rotation relative to the body (multiples of 45/90 deg)	0 = none; 1..40 = the rotation enum in PX4 docs (Sensor Orientation)	0

10 Where things live

scenarios/*.json	aircraft descriptions; control.px4_params_override holds tuned gains
backend/tiltlab/export/gazebo.py	gz sim harness and gain sizing (px4_tuning)
backend/tiltlab/export/gazebo_classic_hitl.py	Classic HITL harness, .params, board label, README
backend/tiltlab/api/gazebo_launch.py	the launch/status/stop
scripts/wsl/tiltlab_gazebo.sh	the launcher (checks, installs, patches, launches, -stop)
scripts/wsl/px4ctl.sh	SITL: takeoff, land, param, log, stop
scripts/wsl/qgc_udp_relay.py	MAVLink relay to QGroundControl on Windows
scripts/px4_board.py	board: status, shell, push, verify, pull-log
scripts/hover_report.py	hover quality from a ulog
scripts/tiltlab_menu.py	make menu, the no-browser front door
docs/sim_stack.md	the architecture map and the HITL findings
docs/hitl_runbook.md	the HITL checklist for this machine
exports/gazebo/, exports/gazebo_hitl/	generated harnesses (not versioned)
third_party/PX4-Autopilot	pinned v1.17.0 source, cite file and line when porting

Further reading, in the order it pays off: PX4 “Basic Concepts” and “Control Allocation”; gz sim “Getting started”; sdformat.org spec (world, model, link, joint); PX4 “Simulation” and “Hardware in the Loop”; usbipd-win README; PX4 “Parameters and Configurations” (the INT32 paragraph above is not in it).